

UNIT 3: CONCURRENCY CONTROL & DEADLOCKS

Course Pattern: TE IT - 2024 Pattern (SPPU)
Status: Publication-Grade Theory Guide

Academic Term: Semester V (2026)
Course Code: PCC-301-ITT

Syllabus Mapping & Unit Structure

The following table outlines the curriculum mapping and priority weightage for Unit 3.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Process Synchronization	Principles of Concurrency, Race Conditions, Critical Section Problem, 3 Core Requirements (Mutual Exclusion, Progress, Bounded Waiting), Software Solutions (Peterson's Algorithm).	High Priority (8-10 Marks Qs)
Hardware & OS Primitives	Hardware Atomic Instructions (TestAndSet, CompareAndSwap), Disabling Interrupts, Mutex Locks, Counting & Binary Semaphores, Monitors.	High Priority (8-10 Marks Qs)
Classical Synchronization	Producer-Consumer Problem, Readers-Writers Problem, Dining Philosophers Problem, IPC Mechanisms (Pipes, Shared Memory, Message Passing).	High Priority (8-10 Marks Qs)
Deadlock Principles & RAG	Principles of Deadlock, 4 Coffman Conditions (Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait), Resource Allocation Graphs (RAG).	High Priority (6-8 Marks Qs)
Deadlock Handling Methods	Deadlock Prevention (Condition Invalidation), Deadlock Avoidance (Banker's Safety & Resource-Request Algorithms), Detection & Recovery (Wait-For Graph).	High Priority (10-12 Marks Qs)

Principles of Concurrency and Process Synchronization

Principles of Concurrency and Race Condition

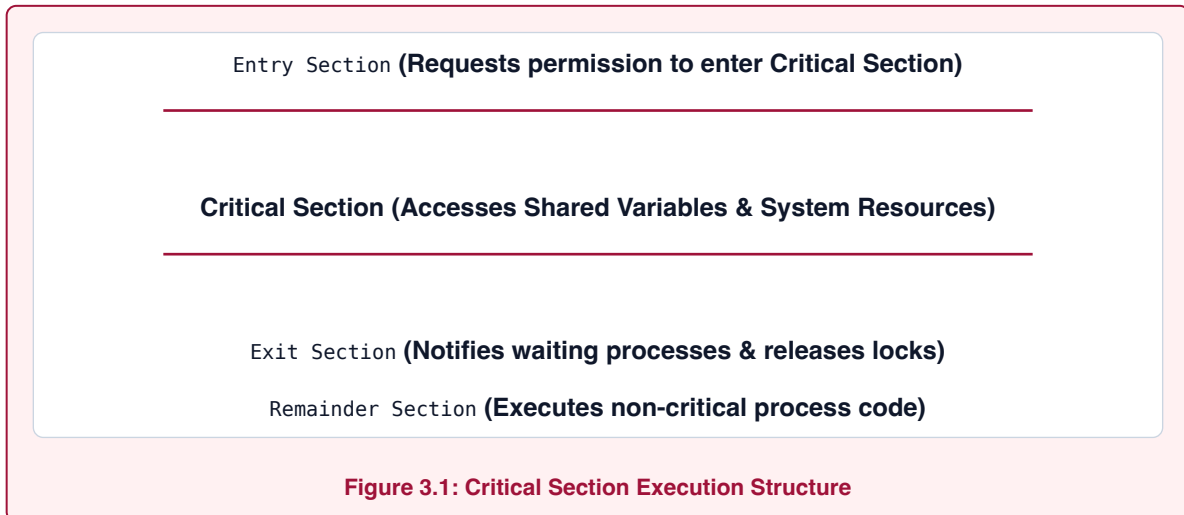
In a concurrent execution environment, multiple processes or threads execute interleaved on a single processor or concurrently across multi-core CPUs. When concurrent processes access and manipulate shared data items simultaneously, the final outcome depends on the precise order of execution, leading to a **Race Condition**.

- Definition of Race Condition:** A flaw in a concurrent system where the output is non-deterministic and depends on the uncontrolled relative timing or sequence of thread execution.
- Printer Spooler Classic Example:**
 - Two processes (P_1 and P_2) want to print a document. Both read the shared variable `in` (pointing to next free slot 7 in print directory).
 - P_1 reads `in = 7`. Before P_1 can store its file name at slot 7, a CPU timer interrupt occurs and context-switches execution to P_2 .
 - P_2 reads `in = 7`, writes its file name at slot 7, increments `in` to 8, and exits.

- When P_1 resumes, it overwrites slot 7 with its own file name, erasing P_2 's print job forever.

The Critical Section Problem and Core Requirements

A **Critical Section** is a segment of code in a process that accesses shared resources (shared variables, memory tables, global files) that must not be concurrently accessed by more than one process.



Three Mandatory Requirements for Valid Solutions

To qualify as a valid solution to the Critical Section Problem, an algorithm must satisfy three strict formal criteria expected by SPPU paper evaluators:

1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other process can be executing in their critical section for that shared resource simultaneously.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter, only those processes not executing in their remainder section can participate in deciding which process enters next. This selection cannot be postponed indefinitely.
3. **Bounded Waiting:** There must be a bound or limit on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter, before that request is granted (prevents starvation).

Source: Stallings, Ch 5, p. 205-225 | Source: Silberschatz, Ch 5, p. 191-215

Synchronization Primitives and Hardware Mechanisms

Hardware-Assisted Synchronization: Atomic Instructions

Software solutions (like Peterson's algorithm) incur high overhead and are difficult to scale on modern out-of-order execution multiprocessors. Modern hardware provides special **Atomic Instructions** executed indivisibly by CPU memory bus locking controllers.

1. **TestAndSet Instruction:** Atomically reads a memory location and sets its value to `true` in a single un-interruptible bus cycle.

```
boolean TestAndSet(boolean *target) {
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
// Mutual Exclusion Usage: while (TestAndSet(&lock)); /* Busy wait */
```

2. **CompareAndSwap (CAS) Instruction:** Atomically compares the value at a memory location with an expected value, updating it to a new value only if they match.

OS Synchronization Support: Semaphores and Mutex Locks

1. **Mutex Lock:** A simplified binary flag (available) designed for mutual exclusion. A thread calls `acquire()` before entering the critical section and `release()` upon leaving.
2. **Semaphore:** An integer variable S accessed exclusively through two standard atomic operations: `wait()` (historically P) and `signal()` (historically V).

Parameter	Counting Semaphore	Binary Semaphore / Mutex
Value Range	Unrestricted integer range (0 to N).	Strictly constrained to binary values (0 or 1).
Resource Management	Controls access to finite multi-instance resource pools (e.g., N database connections).	Controls exclusive single-instance access to a critical section.
Initialization	Initialized to available resource capacity count N .	Initialized to 1 (available) or 0 (locked).
Ownership Rule	Any thread can invoke <code>signal()</code> to release a slot.	Only the acquiring thread should release the mutex lock.

Implementation of Non-Busy Waiting Semaphores

To eliminate CPU-wasting busy-waiting (spinlocks), OS kernels implement semaphores using a system wait queue:

```
typedef struct {
    int value;
    struct process *queue; // List of blocked process PCBs
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        // Add calling process PCB to S->queue
        block(); // Transition process state to Blocked
    }
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        // Remove process P from S->queue
        wakeup(P); // Transition process P state to Ready
    }
}
```

High-Level Language Support: Monitors

A **Monitor** is a high-level language synchronization construct (supported in Java, Concurrent Pascal) that encapsulates private data variables, internal procedure methods, and condition variables within an abstract data type.

- **Implicit Mutual Exclusion:** Only one process can execute inside a monitor procedure at any given instant.
- **Condition Variables:** condition x , y ; accessed via $x.wait()$ (suspends caller process) and $x.signal()$ (resumes one suspended process).

Source: Stallings, Ch 5, p. 226-250 | Source: Silberschatz, Ch 5, p. 225-255

Classical Synchronization Problems and IPC

The Producer-Consumer Problem (Bounded Buffer)

One or more producer processes generate data items placed into a bounded buffer of capacity N , while consumer processes remove items concurrently.

1. Synchronization Semaphores Used:

- **mutex** (Binary Semaphore, $Init = 1$): Ensures mutual exclusion for buffer insertion/removal.
- **empty** (Counting Semaphore, $Init = N$): Tracks number of empty buffer slots.
- **full** (Counting Semaphore, $Init = 0$): Tracks number of populated buffer slots.

2. Algorithmic Implementation:

```
// Producer Process Code
do {
    // produce item in next_produced
    wait(&empty); // Decrement empty slots
    wait(&mutex); // Enter critical section
    buffer[in] = next_produced;
    in = (in + 1) % N;
    signal(&mutex); // Exit critical section
    signal(&full); // Increment full slots
} while (TRUE);

// Consumer Process Code
do {
    wait(&full); // Decrement full slots
    wait(&mutex); // Enter critical section
    next_consumed = buffer[out];
    out = (out + 1) % N;
    signal(&mutex); // Exit critical section
    signal(&empty); // Increment empty slots
    // consume item in next_consumed
} while (TRUE);
```

The Readers-Writers Problem

A shared database is accessed by multiple concurrent processes. **Readers** only read data, while **Writers** modify data.

- Multiple readers can read simultaneously without conflict.
- Writers require exclusive access (no readers or other writers permitted).
- **Solution Primitives:** Binary semaphores rw_mutex ($Init = 1$), $mutex$ ($Init = 1$), and integer $read_count$ ($Init = 0$).

The Dining Philosophers Problem

Five philosophers sit around a circular table with five chopsticks. Each philosopher requires two adjacent chopsticks (left and right) to eat.

- Illustrates allocation of multiple resources among competing processes without causing deadlock or starvation.
- **Deadlock Pitfall:** If all 5 philosophers grab their left chopstick simultaneously, all wait forever for their right chopstick (Circular Wait).

Source: Tanenbaum, Ch 2, p. 120-145 | Source: Silberschatz, Ch 6, p. 257-275

Deadlock Principles and Resource Allocation Graphs

Principles of Deadlock and 4 Coffman Conditions

A **Deadlock** is a set of blocked processes, each holding a physical/logical resource and waiting to acquire a resource held by another process in the set.

The Four Necessary and Sufficient Coffman Conditions

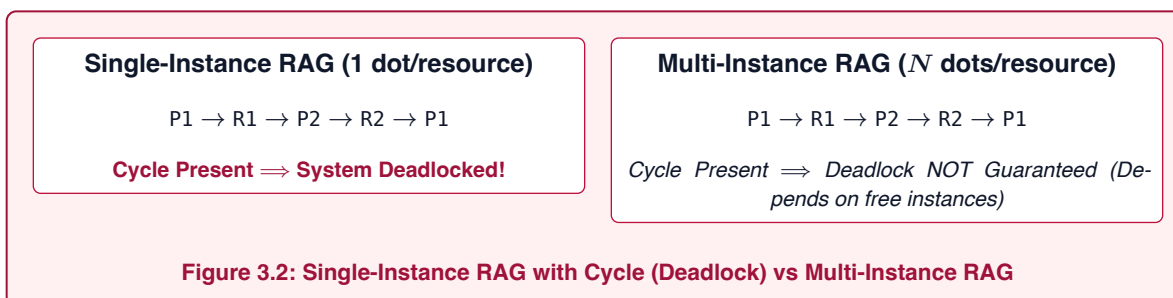
Deadlock can arise if and only if all four **Coffman conditions** hold simultaneously in a system:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode (only one process can use the resource at a time).
2. **Hold and Wait:** A process must be currently holding at least one resource while waiting to acquire additional resources held by other processes.
3. **No Preemption:** Resources cannot be forcibly preempted from a process; they can only be released voluntarily after the process completes its task.
4. **Circular Wait:** A closed chain of processes $\{P_0, P_1, \dots, P_n\}$ exists such that P_0 waits for a resource held by P_1 , P_1 waits for P_2 , and P_n waits for a resource held by P_0 .

Resource Allocation Graphs (RAG)

A **Resource Allocation Graph (RAG)** is a directed graph $G = (V, E)$ used to model system resource allocations.

- **Vertices (V):** Partitioned into Process nodes $P = \{P_1, P_2, \dots, P_n\}$ (represented as Circles) and Resource type nodes $R = \{R_1, R_2, \dots, R_m\}$ (represented as Rectangles containing instance dots).
- **Edges (E):**
 - **Request Edge (P_i to R_j):** Directed edge from process to resource node.
 - **Assignment Edge (R_j to P_i):** Directed edge from specific resource instance dot to process node.



Source: Stallings, Ch 6, p. 265-285 | Source: Silberschatz, Ch 7, p. 315-335

Deadlock Handling Strategies

Overview of Deadlock Strategies

Modern operating systems employ four major strategies to handle deadlocks:

Strategy	Core Mechanism	Advantages	Disadvantages
Ignorance (Ostrich)	Ignore problem completely. Assume deadlocks occur rarely (Used in Linux, Windows).	Zero runtime monitoring overhead.	System hangs if deadlock occurs; requires manual reboot.
Prevention	Restrict resource requests by invalidating at least 1 Coffman condition.	Prevents deadlocks structurally.	Low resource utilization and throughput.
Avoidance	Dynamic tracking of max resource claims via Banker's Algorithm.	High resource utilization.	Requires priori knowledge of max process resource demands.
Detection & Recovery	Periodically run detection algorithms; recover via termination or preemption.	Flexible resource allocation.	High overhead of detection algorithms and lost work.

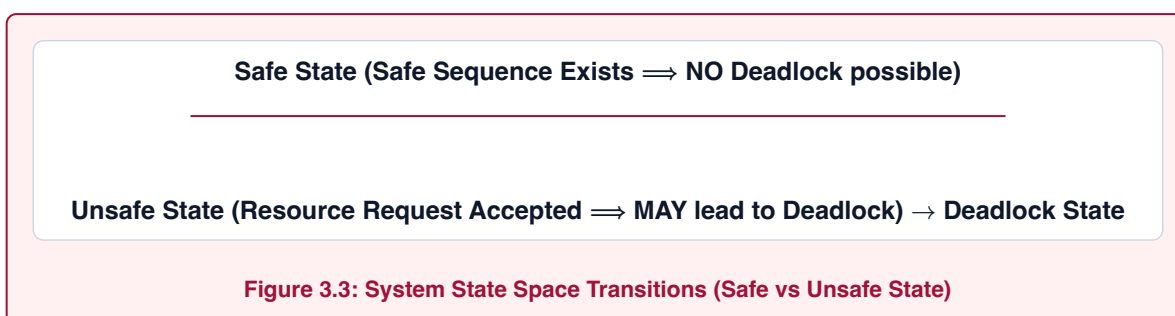
Deadlock Prevention Techniques

Prevents deadlocks by designing system protocols so that at least one Coffman condition can never hold:

1. **Eliminating Mutual Exclusion:** Read-only files can be shared concurrently. However, non-shareable hardware (printers, tape drives) fundamentally require mutual exclusion.
2. **Eliminating Hold and Wait:** Require a process to request and receive all required resources before starting execution, or release current resources before requesting new ones (leads to low resource utilization).
3. **Eliminating No Preemption:** If a process holding resources requests another resource that cannot be immediately assigned, all currently held resources are forcibly preempted.
4. **Eliminating Circular Wait:** Impose a strict total ordering of all resource types $F : R \rightarrow \mathbb{N}$. Processes must request resources strictly in increasing numerical order.

Deadlock Avoidance: Banker's Algorithm

Deadlock avoidance dynamic protocols evaluate resource requests in real-time to guarantee the system remains in a **Safe State**.



Banker's Algorithm Vectors and Matrices

For n processes and m resource types:

- Available[m]: Available instances of each resource type.
- Max[n][m]: Maximum resource demand of process P_i .
- Allocation[n][m]: Currently allocated resource instances to process P_i .
- Need[n][m]: Remaining resource demand of process P_i ($\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$).

1. Safety Algorithm

1. Let $Work = Available$ (vector of length m) and $Finish[i] = FALSE$ for $i = 0, 1, \dots, n - 1$.
2. Find an index i such that: $Finish[i] == FALSE$ AND $Need[i] \leq Work$
3. If no such i exists, go to step 4. Otherwise: $Work = Work + Allocation[i]$ $Finish[i] = TRUE$ Go to step 2.
4. If $Finish[i] == TRUE$ for all i , then the system is in a **Safe State** (the order of processes forms a **Safe Sequence**).

2. Resource-Request Algorithm

When a request vector $Request[i]$ is made by process P_i :

1. If $Request[i] \leq Need[i]$, go to step 2. Else error (exceeded max claim).
2. If $Request[i] \leq Available$, go to step 3. Else P_i must wait (insufficient resources).
3. Pretend to allocate resources to P_i : $Available = Available - Request[i]$ $Allocation[i] = Allocation[i] + Request[i]$ $Need[i] = Need[i] - Request[i]$
4. Run **Safety Algorithm**. If Safe $\rightarrow$ Grant request. Else $\rightarrow$ Rollback allocation and force P_i to wait.

Source: Stallings, Ch 6, p. 286-310 | Source: Silberschatz, Ch 7, p. 336-360

End of Operating Systems Unit 3 Theory Notes — SPPU TE IT 2024 Pattern